Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине «Модели решения задач в ИС»
Тема: «Бинарная классификация с использованием кросс-энтропии (логистическая регрессия)»

Выполнила:
Студент 3 курса
Группы ИИ-26 (2)
Черноиван А.Н.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провести **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Задачи лабораторной работы:

1. Модифицировать программу из ЛР №1:
   - Заменить пороговую функцию активации на сигмоидную.
   - Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
   - Реализовать вычисление градиента $(\hat{y} - e)x_i$ для обновления весов.
2. Реализовать обучение с **фиксированным** шагом (взять значение α, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку $E_s(p)$ на каждой эпохе p.

3. Реализовать обучение с **адаптивным** шагом:
   - На каждой итерации (после подачи одного образа) вычислять $\alpha(t)$ (формула (2.36)).
   - Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
   - Сохранять значения $E_s(p)$ для построения графика.
4. Провести сравнительный анализ:
   - Построить на одном графике кривые сходимости $E_s(p)$ для четырёх конфигураций:
     - MSE + фиксированный шаг (данные из ЛР №1);
     - MSE + адаптивный шаг (данные из ЛР №2);
     - BCE + фиксированный шаг;
     - BCE + адаптивный шаг.
   - Зафиксировать число эпох, необходимое для достижения заданной точности $E_s \leq E_e$ (взять E_e из ЛР №2, например 0.01).
5. Визуализировать результаты:
   - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
   - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
   - Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.
6. Реализовать режим функционирования:
   - Пользователь вводит координаты x_1, x_2.
   - Сеть вычисляет вероятность $\hat{y}$ и, применяя порог, определяет предсказанный класс.
   - Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).
7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.
8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

10.

x_1	x_2	e
6	6	0
-6	6	0
6	-6	1
-6	-6	0

Код программы:

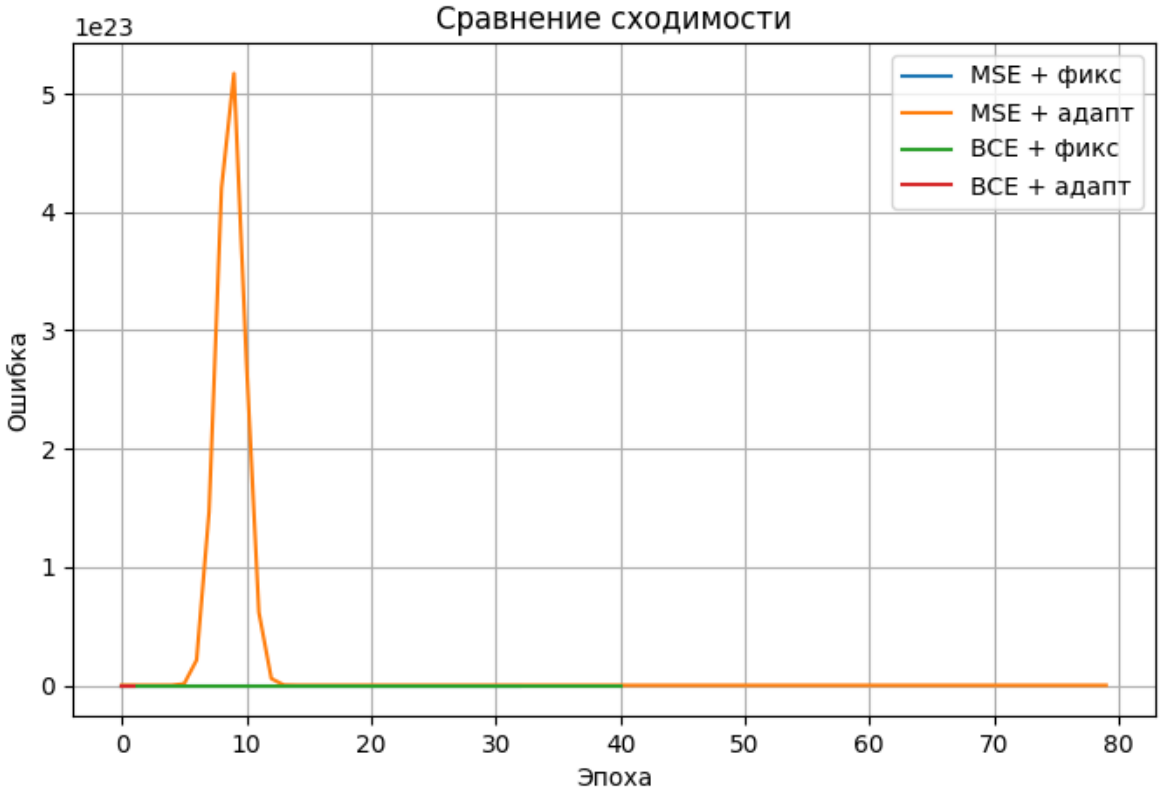
```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [6, 6],
    [-6, 6],
    [6, -6],
    [-6, -6]
], dtype=float)

E = np.array([1 if x[0] > 0 else 0 for x in X], dtype=float)

epochs = 100
Ee = 0.01

def sigmoid(S):
    return 1 / (1 + np.exp(-S))

def BCE_loss(y, e):
    eps = 1e-9
    return -(e*np.log(y+eps) + (1-e)*np.log(1-y+eps))

def train_MSE_fixed(X, E, eta):
    w = np.random.randn(2)
    T = np.random.randn()
    errors = []

    for epoch in range(epochs):
        mse = 0
        for x, e in zip(X, E):
            S = np.dot(w, x) - T
            y = S
            error = e - y
            mse += error**2

            w += eta * error * x
            T -= eta * error

        errors.append(mse/len(X))
        if errors[-1] <= Ee:
            break
```

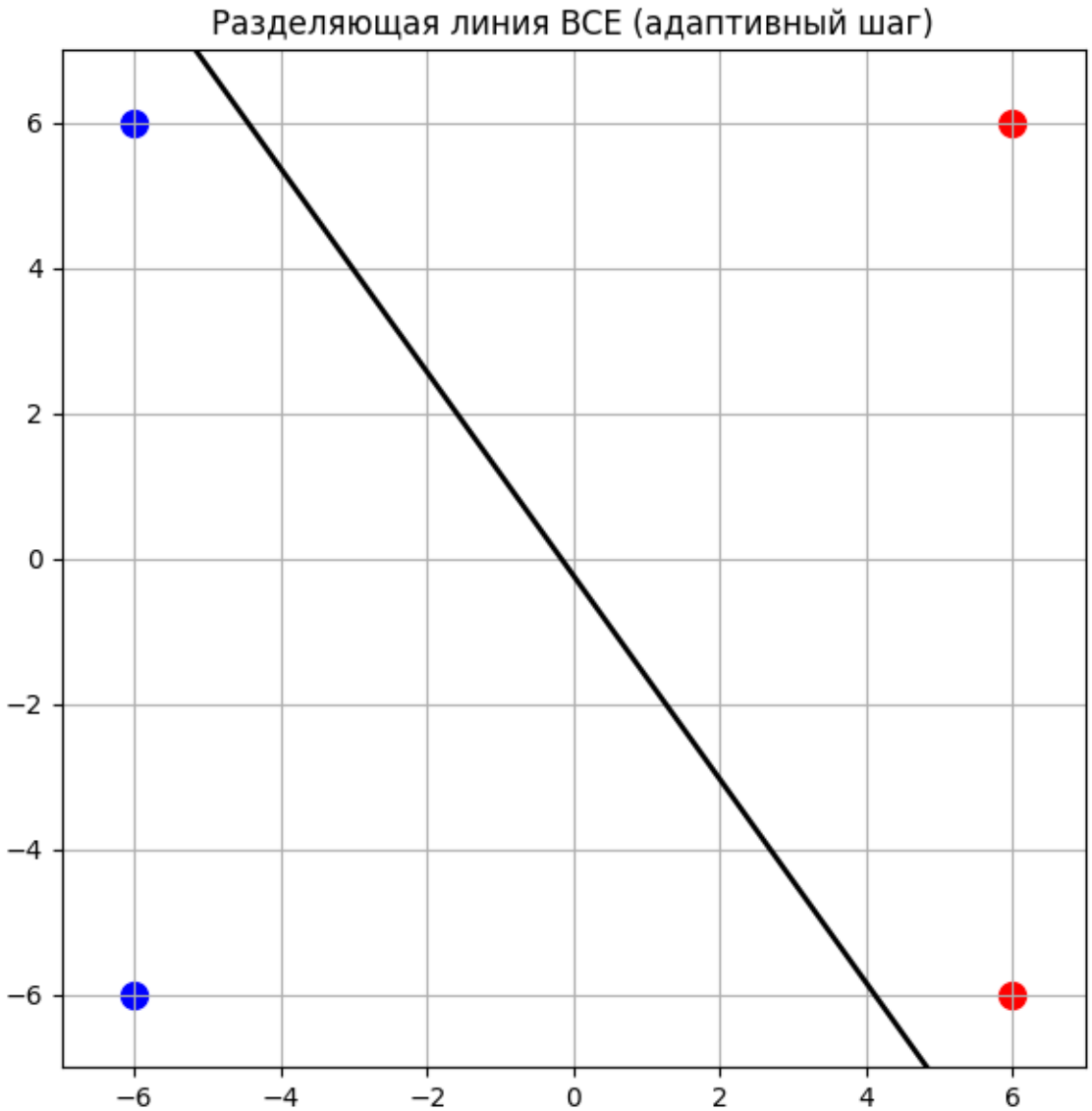
```
    return w, T, errors

def train_MSE_adaptive(X, E):
    w = np.random.randn(2)
    T = np.random.randn()
    errors = []
    t = 1

    for epoch in range(epochs):
        mse = 0
        for x, e in zip(X, E):

            eta = 0.5 / t
            t += 1

            S = np.dot(w, x) - T
            y = S
            error = e - y
            mse += error**2

            w += eta * error * x
            T -= eta * error

        errors.append(mse/len(X))
        if errors[-1] <= Ee:
            break

    return w, T, errors

def train_BCE_fixed(X, E, eta):
    w = np.random.randn(2)
    T = np.random.randn()
    errors = []

    for epoch in range(epochs):
        ce = 0
        for x, e in zip(X, E):

            S = np.dot(w, x) - T
            y = sigmoid(S)

            ce += BCE_loss(y, e)

            grad = (y - e)
            w -= eta * grad * x
            T += eta * grad

        errors.append(ce/len(X))
        if errors[-1] <= Ee:
            break

    return w, T, errors


def train_BCE_adaptive(X, E):
    w = np.random.randn(2)
```

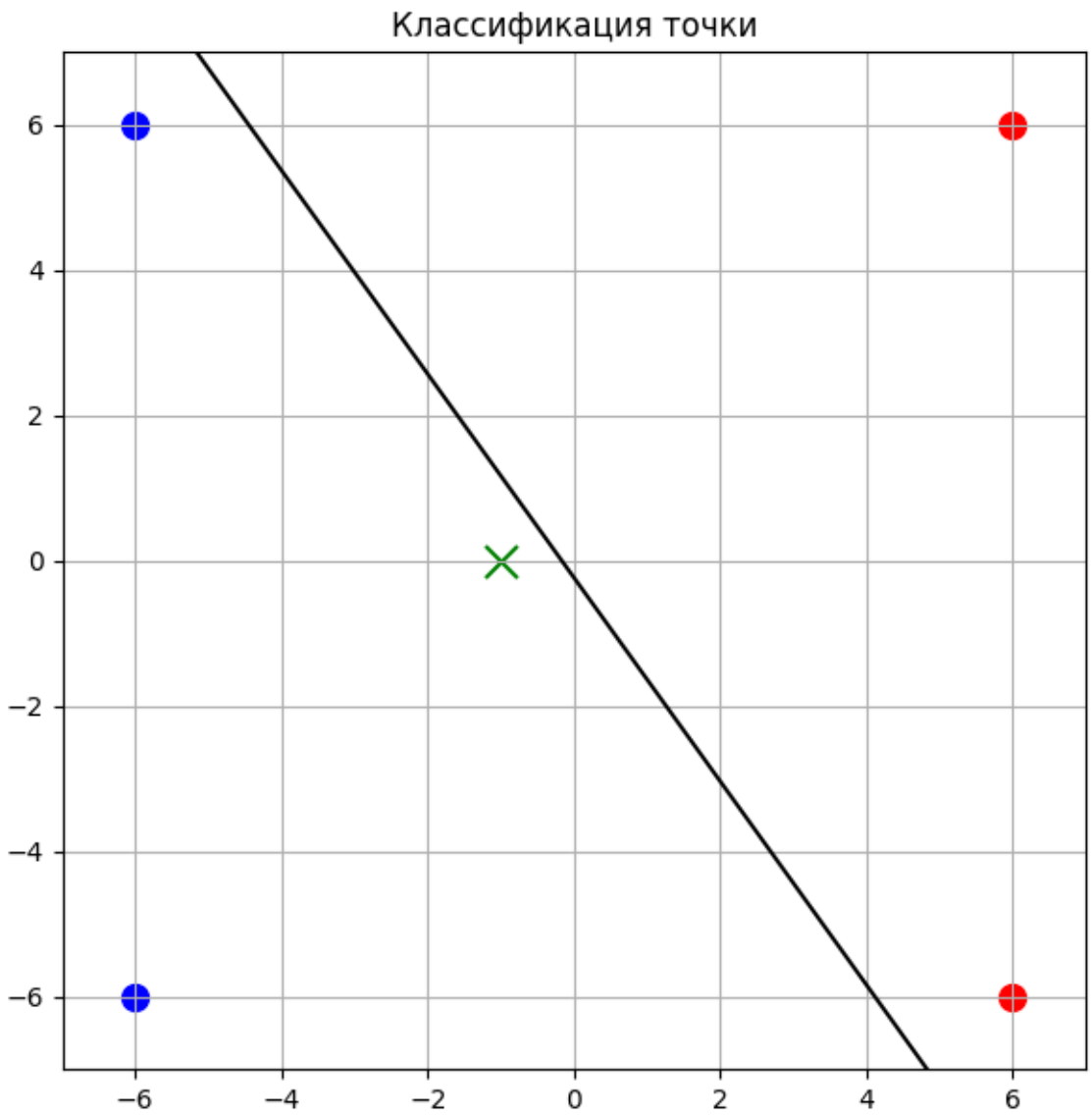
```
    T = np.random.randn()
    errors = []
    t = 1

    for epoch in range(epochs):
        ce = 0
        for x, e in zip(X, E):

            eta = 1 / t
            t += 1

            S = np.dot(w, x) - T
            y = sigmoid(S)

            ce += BCE_loss(y, e)

            grad = (y - e)
            w -= eta * grad * x
            T += eta * grad

        errors.append(ce/len(X))
        if errors[-1] <= Ee:
            break

    return w, T, errors


w1, T1, e1 = train_MSE_fixed(X, E, 0.01)
w2, T2, e2 = train_MSE_adaptive(X, E)
w3, T3, e3 = train_BCE_fixed(X, E, 0.01)
w4, T4, e4 = train_BCE_adaptive(X, E)

plt.figure(figsize=(8,5))
plt.plot(e1, label="MSE + фикс")
plt.plot(e2, label="MSE + адапт")
plt.plot(e3, label="BCE + фикс")
plt.plot(e4, label="BCE + адапт")
plt.xlabel("Эпоха")
plt.ylabel("Ошибка")
plt.title("Сравнение сходимости")
plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(7,7))

for i in range(len(X)):
    color = "red" if E[i] == 1 else "blue"
    plt.scatter(X[i,0], X[i,1], color=color, s=100)

x_line = np.linspace(-7, 7, 300)
y_line = (T4 - w4[0]*x_line) / w4[1]
plt.plot(x_line, y_line, 'k', linewidth=2)

plt.xlim(-7, 7)
plt.ylim(-7, 7)
```

```
plt.grid()
plt.title("Разделяющая линия ВСЕ (адаптивный шаг)")
plt.show()


def classify(x1, x2):

    S = w4[0]*x1 + w4[1]*x2 - T4
    y_prob = sigmoid(S)
    y_class = 1 if y_prob >= 0.5 else 0

    print("\n========== РЕЗУЛЬТАТ ==========")
    print(f"Вероятность класса 1: {y_prob:.4f}")
    print(f"Предсказанный класс: {y_class}")
    print("================================\n")

    plt.figure(figsize=(7,7))

    for i in range(len(X)):
        color = "red" if E[i] == 1 else "blue"
        plt.scatter(X[i,0], X[i,1], color=color, s=100)

    plt.plot(x_line, y_line, 'k')
    plt.scatter(x1, x2, color="green", s=150, marker="x")

    plt.xlim(-7, 7)
    plt.ylim(-7, 7)
    plt.grid()
    plt.title("Классификация точки")
    plt.show()

while True:
    x1 = input("x1 (exit для выхода): ")
    if x1 == "exit":
        break
    x2 = input("x2: ")

    try:
        classify(float(x1), float(x2))
    except:
        print("Ошибка ввода")
```

Результат программы:


Сравнение сходимости
1e23
MSE + фикс
MSE + адапт
BCE + фикс
BCE + адапт
Ошибка
Эпоха
0
1
2
3
4
5
0
10
20
30
40
50
60
70
80


Разделяющая линия BCE (адаптивный шаг)
6
4
2
0
−2
−4
−6
−6
−4
−2
0
2
4
6



Критерий сравнения	MSE + фиксированный шаг	MSE + адаптивный шаг	BCE + фиксированный шаг	BCE + адаптивный шаг
Скорость сходимости	Средняя	Немного выше, чем при фиксированном шаге	Быстрее, чем MSE	Наиболее быстрая
Количество эпох до достижения Ee	Больше	Меньше, чем у MSE фикс.	Меньше, чем у MSE	Минимальное
Поведение градиента	Градиент уменьшается при приближении к минимуму	Частично компенсируется адаптацией шага	Градиент сохраняет информативность даже при уверенных ошибках	Быстро корректирует крупные ошибки
Теоретическое объяснение	MSE плохо согласуется с логистической функцией активации, градиенты могут “затухать”	Адаптация шага ускоряет движение в плоских областях	BCE соответствует максимуму правдоподобия для бинарной классификации	Адаптивный шаг усиливает преимущества BCE

Стабильность обучения	Стабильно, но медленно	Более устойчиво к плохому выбору η	Хорошая	Высокая
Итоговая разделяющая линия	Может немного отличаться	Ближе к оптимальной	Более теоретически обоснованная	Практически совпадает с BCE фикс., но достигается быстрее
Причина возможных расхождений линий	Из-за различий в функции потерь оптимизируется разный функционал	Шаг влияет на траекторию движения в пространстве весов	Минимизируется логарифмическая вероятность ошибки	Быстрая сходимость может привести к немного иной траектории

Вывод: для задачи бинарной классификации более предпочтительной является функция потерь **BCE**, так как она теоретически согласована с логистической моделью и обеспечивает более быструю и устойчивую сходимость. Использование **MSE** допустимо, однако обучение происходит медленнее из-за эффекта затухания градиента. Применение адаптивного шага улучшает скорость сходимости для обеих функций потерь, но наибольший эффект наблюдается при использовании BCE.